# AI Sketch — Reconnaissance de dessins à la main

Projet du cours **ANI-2083 – Modèle de la SAT et IA dans les Arts Technologiques** (Mini-projet 2 — Semaine 2)

**Auteur** : [Steve Akoa / Groupe 4] **Date** : 14 octobre 2026 **Enseignant** : Dr. Aurelle TCHAGNA

---

## 1. Description

**AI Sketch** est une application desktop qui permet à l'utilisateur de **dessiner un objet à la souris** dans une zone blanche, puis de demander à un modèle d'intelligence artificielle de **reconnaître automatiquement** ce qui a été dessiné.

Le modèle est un **réseau de neurones convolutif (CNN)** entraîné sur un sous-ensemble du dataset **Google Quick, Draw!**.

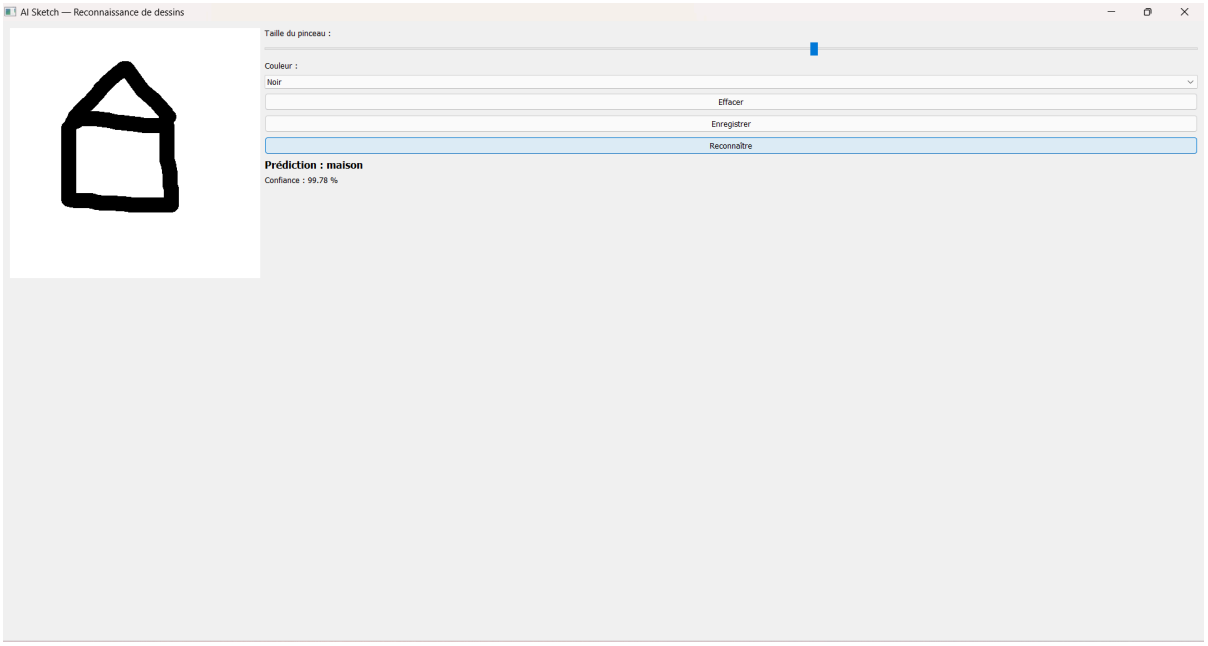
### Catégories reconnues

L'application distingue **5 classes** :

- 🏠 `maison`
- 🌳 `arbre`
- 🚗 `voiture`
- 🐱 `chat`
- 🌞 `soleil`

---

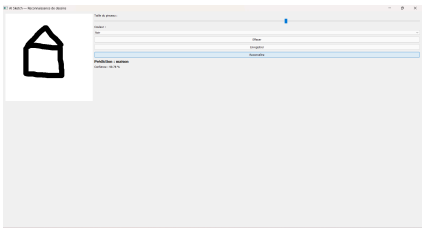
## 2. Démonstration



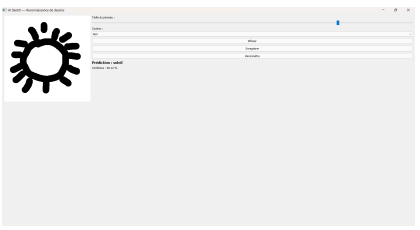

**Exemple** : un dessin de maison simple (carré + triangle, pinceau épais) correctement identifié avec **99,78 %** de confiance.

D'autres exemples sont disponibles dans le dossier `screenshots/` :

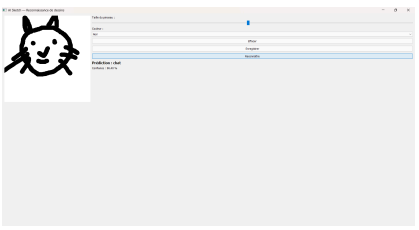
- `test_04_soleil.png` — soleil reconnu (96,12 %)
- `test_05_chat.png` — chat reconnu (96,40 %)
- `test_07_voiture.png` — voiture reconnue (95,04 %)
- `test_08_arbre.png` — arbre reconnu (97,28 %)
- `test_06_soleil_courbe.png` — cas de confusion (soleil classé `chat`)



Maison (99,78 %)



Soleil (96,12 %)



Chat (96,40 %)

---

## 3. Structure du projet

```
ai_sketch/
│
├── app.py                      # Application principale (interface PyQt5)
├── train_model.py              # Script d'entraînement du CNN
├── predict.py                  # Prédiction sur un dessin
├── download_data.py            # Téléchargement du dataset Quick Draw
├── preview.py                  # Aperçu visuel des images du dataset
├── requirements.txt            # Dépendances Python
├── README.md                   # Ce fichier
├── RAPPORT.md                  # Rapport de projet
├── RAPPORT.pdf                 # Rapport (version PDF)
│
├── data/                       # Dataset (2500 images, 5 classes)
│   ├── arbre/                  #   500 images (.png)
│   ├── chat/                   #   500 images (.png)
│   ├── maison/                 #   500 images (.png)
│   ├── soleil/                 #   500 images (.png)
│   └── voiture/                #   500 images (.png)
│
├── model/                      # Modèle entraîné
│   ├── sketch_model.h5         #   Réseau de neurones (format HDF5)
│   └── classes.txt             #   Liste des classes (ordre des labels)
│
├── screenshots/                # Captures d'écran des tests
│   ├── test_01_maison_epais.png
│   ├── test_02_maison_fin.png
│   ├── test_03_maison_detaillee.png
│   ├── test_04_soleil.png
│   ├── test_05_chat.png
│   ├── test_06_soleil_courbe.png
│   ├── test_07_voiture.png
```

```
│   ├── test_08_arbre.png
│   ├── test_09_zigzag1.png
│   ├── test_10_zigzag2.png
│   └── test_11_etoile.png
│
├── cache_npy/                  # Cache des fichiers .npy téléchargés
│   ├── house.npy
│   ├── tree.npy
│   ├── car.npy
│   ├── cat.npy
│   └── sun.npy
│
└── venv/                       # Environnement virtuel Python 3.11
```

### Légende

| Dossier / fichier | Rôle |
|---|---|
| `app.py` | Point d'entrée — lance la fenêtre PyQt5 |
| `train_model.py` | Charge `data/` et entraîne le CNN |
| `predict.py` | Fonction de prédiction utilisée par `app.py` |
| `download_data.py` | Télécharge les 5 fichiers `.npy` depuis Google |
| `preview.py` | Affiche une grille d'aperçu du dataset |
| `data/` | Dataset final prêt à l'emploi (2500 PNG) |
| `model/` | Résultat de l'entraînement (modèle + labels) |
| `screenshots/` | Captures d'écran pour le rapport |
| `cache_npy/` | Cache brut des `.npy` téléchargés (~150 Mo) |
| `venv/` | Environnement virtuel (non livré, à recréer avec `py -3.11 -m venv venv`) |

---

## 4. Installation

### Prérequis

- **Python 3.11** (obligatoire — TensorFlow 2.15 ne supporte pas Python 3.13)
- Windows 10/11, Linux ou macOS
- Environ 500 Mo d'espace disque

### Étapes

#### 1. Télécharger et installer Python 3.11

Lien officiel : https://www.python.org/ftp/python/3.11.7/python-3.11.7-amd64.exe

⚠️ **Cocher impérativement** « Add python.exe to PATH » pendant l'installation.

**2. Créer un environnement virtuel**

```
cd C:\ai_sketch
py -3.11 -m venv venv
venv\Scripts\activate     # Windows
# ou
source venv/bin/activate   # Linux/Mac
```

**3. Installer les dépendances**

```
pip install --upgrade pip
pip install -r requirements.txt
```

**4. (Important — si la connexion est instable)**

Pour télécharger TensorFlow manuellement :

- Aller sur https://pypi.org/project/tensorflow-intel/2.15.0/#files
- Chercher `tensorflow_intel-2.15.0-cp311-cp311-win_amd64.whl` (~287 Mo)
- Installer localement :

```
pip install "chemin/vers/tensorflow_intel-2.15.0-cp311-cp311-win_amd64.whl"
```

---

# 5. Utilisation

## Étape 1 — Télécharger le dataset (facultatif si `data/` est déjà rempli)

```
python download_data.py
```

Télécharge 500 images par classe depuis Google Quick, Draw! dans `data/`.

## Étape 2 — Entraîner le modèle

```
python train_model.py
```

Durée : **3 à 10 minutes sur CPU**. À la fin, le modèle est enregistré dans `model/sketch_model.h5`.

Résultat attendu :

```
Epoch 15/15
loss: 0.0270 - accuracy: 0.9945 - val_loss: 0.1625 - val_accuracy: 0.9520
Modèle enregistré : model/sketch_model.h5
```

### Étape 3 — Lancer l'application

```
python app.py
```

La fenêtre **AI Sketch** s'ouvre. Dessinez dans la zone blanche, puis cliquez sur **Reconnaître**.

---

## 6. Fonctionnalités

| Fonctionnalité | Statut |
|---|---|
| Zone de dessin à la souris | ✅ |
| Choix de la taille du pinceau (1 à 40) | ✅ |
| Choix de la couleur (Noir / Rouge / Bleu / Vert) | ✅ |
| Bouton **Effacer** | ✅ |
| Bouton **Enregistrer** (export PNG) | ✅ |
| Bouton **Reconnaître** (prédiction) | ✅ |
| Affichage de la classe prédite | ✅ |
| Affichage du score de confiance | ✅ |
| Gestion du cas « modèle manquant » | ✅ |

## 6bis. Bonus créativité

L'application intègre **4 bonus créativité** supplémentaires :

### 🎨 Filtres artistiques

Appliqués en un clic grâce à OpenCV :

- **Crayon** (`cv2.divide` + flou gaussien) → effet croquis
- **Cartoon** (`bilateralFilter` + `adaptiveThreshold`) → contours + aplats
- **Pastel** (`cv2.stylization`) → effet pictural
- Bouton ↺ pour restaurer le dessin original

### 🖼️ Galerie de dessins

- Bouton 💾 **Ajouter à la galerie** : enregistre le dessin dans `galerie/`
- Panneau de **miniatures** à droite de la fenêtre

- Double-clic sur une miniature pour la **recharger** dans le canvas
- Bouton 🔄 **Rafraîchir** pour mettre à jour la liste

### 📜 Historique des prédictions

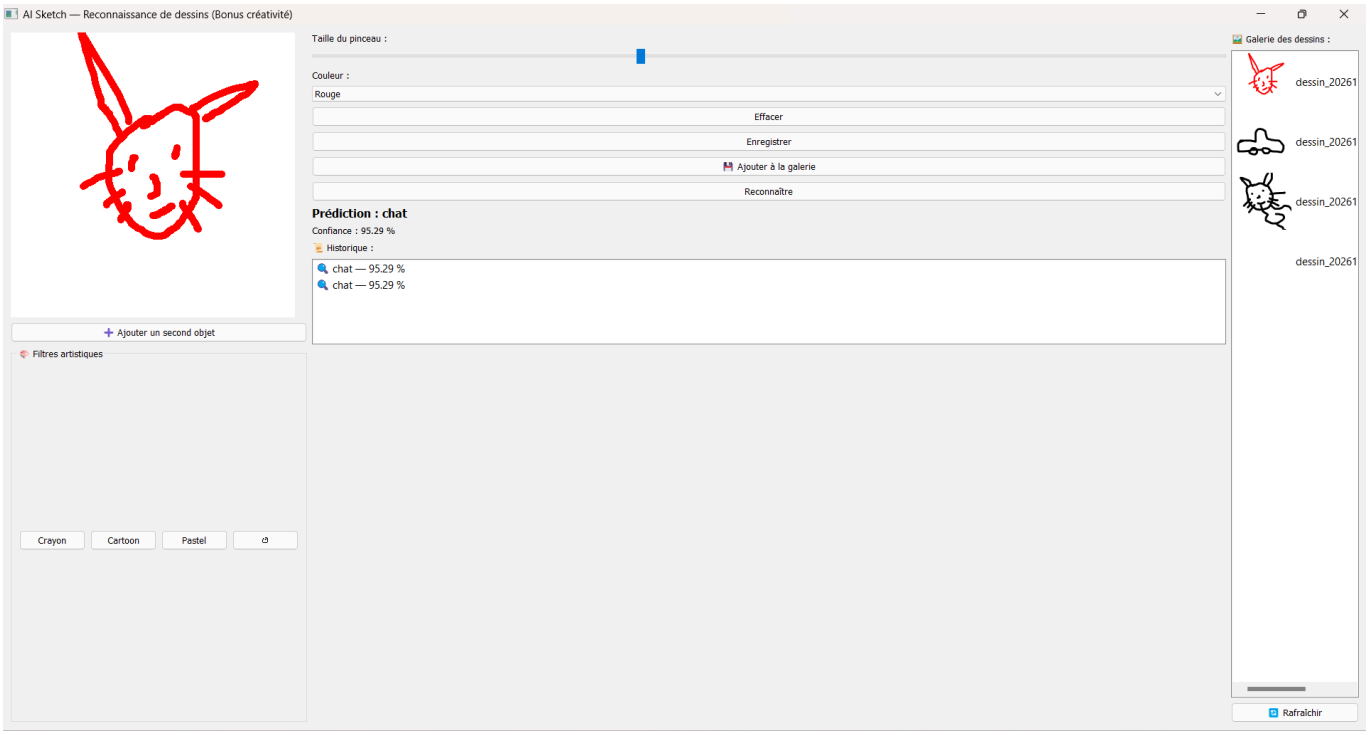
Panneau conservant la liste des prédictions avec leur score de confiance.

### ➕ Second objet

Un bouton ➕ **Ajouter un second objet** permet d'enchaîner plusieurs dessins, tout en conservant l'historique.

### 🎨 Palette étendue

8 couleurs disponibles : Noir, Rouge, Bleu, Vert, Orange, Violet, Rose, Marron.
Taille du pinceau par défaut : **15 px** (meilleure précision).




---

# 7. Résultats des tests

## 7.1 Protocole

Nous avons testé l'application avec **11 dessins réalisés en temps réel**, en faisant varier :

- le **type d'objet** (5 classes + objets hors-catégories),
- la **taille du pinceau** (pour observer son impact sur la confiance).

## 7.2 Tests sur les 5 classes

| # | Dessin | Prédiction | Confiance | Verdict |
|---|---|---|---|---|
| 1 | Maison (pinceau épais) | `maison` | **99,78 %** | ✅ |

| # | Dessin | Prédiction | Confiance | Verdict |
|---|---|---|---|---|
| 2 | Maison (pinceau fin) | `maison` | 60,66 % | ✅ |
| 3 | Maison détaillée (porte + fenêtre) | `maison` | **99,86 %** | ✅ |
| 4 | Soleil (rayons droits) | `soleil` | 96,12 % | ✅ |
| 5 | Chat (oreilles + moustaches) | `chat` | 96,40 % | ✅ |
| 6 | Soleil (rayons courbés) | `chat` | 49,47 % | ❌ |
| 7 | Voiture (carrosserie + roues) | `voiture` | 95,04 % | ✅ |
| 8 | Arbre (tronc + feuillage) | `arbre` | 97,28 % | ✅ |

**Taux de réussite** : **7 / 8** (87,5 %).

### 7.3 Tests sur des dessins hors-catégories

| # | Dessin | Prédiction | Confiance |
|---|---|---|---|
| 9 | Zigzag montant | `chat` | 94,19 % |
| 10 | Zigzag horizontal | `voiture` | 60,24 % |
| 11 | Étoile | `soleil` | 32,02 % |

👉 Le modèle **force** toujours une réponse parmi les 5 classes, même pour des dessins hors-catégories.

### 7.4 Observation importante : influence de la taille du pinceau

Nous avons observé que **la taille du pinceau a un impact direct sur la confiance du modèle** :

| Taille du pinceau | Confiance observée |
|---|---|
| **Épais** (15–20 px) | ✅ Très haute (95–99 %) |
| **Moyen** (10–12 px) | 🟡 Moyenne (60–95 %) |
| **Fin** (5–8 px) | ⚠️ Basse (30–60 %) |

**Explication** : ce comportement vient de deux facteurs :

1. **Le seuillage binaire** (`cv2.threshold` dans `predict.py`) transforme les pixels en noir/blanc. Un trait épais reste solide après seuillage, un trait fin peut se fragmenter.
2. **La cohérence avec le dataset Quick Draw** : les utilisateurs de Quick Draw dessinent avec des traits larges. Le modèle a donc appris des formes « pleines ». Un trait épais s'en rapproche, un trait fin s'en éloigne.

## Cette observation confirme la **sensibilité du modèle au style de dessin** plutôt qu'à la forme elle-même.

## 8. Limites et améliorations possibles

### Limites observées

- **Sensibilité au style** : un trait épais ou irrégulier baisse la confiance.
- **Confusion entre classes proches** : un soleil aux rayons courbés a été classé `chat`.
- **Pas de classe « inconnu »** : un dessin hors-catégorie est toujours classé dans une des 5 classes.

### Améliorations envisagées

- **Data augmentation** (rotation, épaisseur, bruit) pour rendre le modèle robuste au style.
- **Ajout d'une classe** `unknown` entraînée sur des dessins aléatoires.
- **Plus d'images par classe** (au lieu de 500, passer à 2000).
- **CNN plus profond** ou transfer learning avec MobileNet.

---

## 9. Technologies utilisées

| Bibliothèque | Version | Rôle |
|---|---|---|
| Python | 3.11.7 | Langage principal |
| PyQt5 | 5.15.10 | Interface graphique |
| TensorFlow | 2.15.0 | Réseau de neurones |
| Keras | 2.15.0 | API haut niveau |
| OpenCV | 4.9.0.80 | Traitement d'images |
| NumPy | 1.26.4 | Calcul numérique |
| Pillow | 10.2.0 | Manipulation d'images |
| Matplotlib | 3.8.3 | Aperçu du dataset |
| scikit-learn | 1.4.2 | Split train/test |

---

## 10. Licence et données

- **Code** : usage strictement pédagogique. '- **Datasetimages** : Google Quick, Draw! (licence CC BY 4.0). Source : https://github.com/googlecreativ elab/quickdraw-dataset

28---

## 11. Auteur

**Nom** : [Steve Akoa / Groupe 4] **Cours** : ANI-2083 – Modèle de la SAT et IA dans les Arts Technologiques **Date de remise** : mardi 20 octobre 2026